Interface Control Document (ICD)

GG-GRK High-Level Control — Firmware V1.5.1

Field	Value
Document type	Interface Control Document (as-built)
System	GG-GRK Generic Robotic Kit — High-Level Control (HLC)
Configuration item	Firmware V1.5.1

---

1. Purpose and scope

This document describes **the interface as it actually exists**: every message the HLC can receive, every message and signal it emits, the meaning and units of every data element, and the mechanisms available for detecting faults.

It is written to be **self-contained** — a client can be implemented from this document alone, without reading the firmware source.

---

2. Transport and session

Key property: the HLC is passive on the network. It originates exactly one kind of unsolicited traffic — the OCU reachability probe (section 5.2). There is **no telemetry push, no broadcast, no publish/subscribe**. All data is obtained by polling.

2.1 Connecting

Property	Value
Protocol	HTTP/1.1 over TCP
Port	80
Method	`POST`
Path	`/`
Request body	JSON object, UTF-8
Required headers	`Content-Length`
Response body	JSON object
Connection reuse	**Not supported** — the HLC sends `Connection: close` and closes after every response

A client must open a **new TCP connection for every request**. The board has 8 hardware sockets and services up to 4 connections per main-loop pass.

2.2 Access control

Two independent gates, applied in this order:

1. **Source IP whitelist** — up to 3 addresses. A non-listed source gets HTTP 403 and the connection is closed. This is checked before anything else, including before parsing the body.
2. **Session token** — required on every command except `login`.

2.3 Session lifecycle

```None
  login(user,pass) ──> token (16 chars, A–Z)
        │
        ├─ token accompanies every subsequent request
        ├─ each accepted request refreshes the token's idle timer
        └─ token expires after 300 000 ms (5 min) of no use  ──>
HTTP 401
```

- Up to **5** concurrent sessions. A 6th login evicts the least-recently-used.
- An expired or unknown token yields HTTP 401 `"Invalid or expired token."`; the client should re-`login` and retry.
- Tokens do not survive a reboot.

---

3. What the HLC can receive

Twelve commands. Every request is a JSON object with a `type` field; every command except `login` also requires `token`.

3.1 Command index

`type`	**Purpose**	**Response**	**Side effect**
`login`	Authenticate	`login_result`	—
`get_status`	All telemetry	Status document (section 6)	—
`get_overview`	Power / IO / safety / OCU	`overview`	—
`get_imu`	Inertial + orientation	`imu`	—
`get_gps`	GNSS	`gps`	—
`get_config`	Identity + configuration	`config`	—
`set_relay`	Command a relay	Status document	Relay changes
`set_io_led`	Set IO LED colour	Status document	LED changes
`set_internal_led`	Set MCU LED	Status document	LED changes
`reset_led_control`	Resume automatic LED	Status document	LED → OFF, auto
`set_ip_config`	Network configuration	`success`/`message`	**Reboots after 2 s**
`set_imu_axis_map`	IMU axis assignment	`success`/`message`	Reboot recommended

3.2 `login`

```None
{ "type": "login", "user": "admin", "pass": "1234" }
```

Success:

```None
{ "type": "login_result", "success": true, "token":
"QWERTYUIOPASDFGH" }
```

Failure (HTTP 200):

```None
{ "type": "login_result", "success": false, "code": "E-110",
"message": "Invalid username" }
```

`message` is `"Invalid username"` (`E-110`) or `"Invalid password"` (`E-111`). The token is 16 characters, uppercase `A–Z` only.

3.3 `get_status`

```None
{ "type": "get_status", "token": "<token>" }
```

Returns the full status document — see section 6. This is the union of the four category commands plus top-level fields.

3.4 `get_overview` / `get_imu` / `get_gps` / `get_config`

```None
{ "type": "get_overview", "token": "<token>" }
```

Each returns `type` set to `"overview"`, `"imu"`, `"gps"` or `"config"` respectively, followed by that category's fields at the **top level** of the response (not nested). Field lists are in section 6.2–6.5.

All fields of these responses also appear inside the corresponding `get_status` category object.

3.5 `set_relay`

```None
{ "type": "set_relay", "token": "<token>", "relay_id": 0,
"state": true }
```

Field	Type	Range
`relay_id`	integer	0–3
`state`	boolean	—

Returns the full status document. Out-of-range `relay_id` → HTTP 200 with `"Invalid relay number"`.

Relay 0 and 1 auto-release to `false` 5 000 ms after being set `true`. Relays 2 and 3 latch.

Index	Function
0	Power cut-off, internal Ethernet switch (hard reset) — auto-release 5 s
1	Power cut-off, internal computer (hard reset) — auto-release 5 s
2	Power enable 13.8 V/GND, J16 pin 1 — latching
3	Power enable 13.8 V/GND, J16 pin 2 — latching

3.6 `set_io_led`

```None
{ "type": "set_io_led", "token": "<token>", "color": "GREEN" }
```

`color` ∈ `"OFF"`, `"GREEN"`, `"RED"`, `"ORANGE"`, `"AUTO"`.

Any colour other than `AUTO` engages **manual override**, suspending the automatic annunciation of section 5.3 until `AUTO` or `reset_led_control` is sent. Invalid colour → `"Invalid LED color"`. Returns the full status document.

3.7 `set_internal_led`

```None
{ "type": "set_internal_led", "token": "<token>", "state": true }
```

Returns the full status document.

3.8 `reset_led_control`

```None
{ "type": "reset_led_control", "token": "<token>" }
```

Clears manual override and drives the IO LED OFF; automatic control resumes on the next update. Returns the full status document.

3.9 `set_ip_config`

```None
{
  "type": "set_ip_config", "token": "<token>",
  "controller_ip": "192.168.1.198",
  "whitelist_ips": ["192.168.1.20", "192.168.1.169",
"192.168.1.33"]
}
```

`whitelist_ips` accepts up to **3** entries; more than 3 is rejected. Success:

```None
{ "success": true, "message": "IP configuration updated. Board will reboot." }
```

Sequence: the response is transmitted, the connection is closed, then the board reboots after **2 000 ms**. The client must reconnect at the new address. A malformed address yields "`Invalid IP address or whitelist entry provided.`" and no reboot.

3.10 `set_imu_axis_map`

None

```
{
  "type": "set_imu_axis_map", "token": "<token>",
  "pitch_axis": "X", "roll_axis": "Y", "yaw_axis": "Z",
  "pitch_invert": false, "roll_invert": false, "yaw_invert":
false
}
```

Field	Type	Notes
`pitch_axis`, `roll_axis`, `yaw_axis`	string	`"X"`, `"Y"` or `"Z"`, **case-sensitive**. All three must differ.
`pitch_invert`, `roll_invert`, `yaw_invert`	boolean	Optional, default `false`. Negates that axis.

Success:

None

```
{ "success": true, "message": "IMU axis map set to: pitch=X
roll=Y yaw=Z", "reboot_required": true }
```

Errors: `"Invalid IMU axis (expected X, Y or Z)"`, or `"Pitch, roll and yaw must each use a different axis"`.

The setting persists in flash. **Reboot after changing it** — the startup calibration offsets are captured through the map.

---

4. What the HLC sends

4.1 API responses

One JSON response per request, then the connection closes. Response shapes:

Shape	Commands
Full status document (section 6)	`get_status`, `set_relay`, `set_io_led`, `set_internal_led`, `reset_led_control`
Category document	`get_overview`, `get_imu`, `get_gps`, `get_config`
`login_result`	`login`
`success` + `message`	`set_ip_config`, `set_imu_axis_map`
`error` + `code` + `message`	any failure

4.2 OCU reachability probe — the only unsolicited network traffic

Property	Value
Target	the whitelisted IP whose **final octet is 169**
Transport	UDP, source port 5001 → destination port 5001
Payload	1 byte, value 0, **content is meaningless**
Rate	every 1 000 ms while reachable; every 10 000 ms while unreachable
Suppressed by	any traffic already arriving from the OCU

The OCU does not need to listen on port 5001 or reply. The probe exists solely to force an ARP resolution; the ARP reply is the reachability signal. The OCU's network stack answering ARP is sufficient — no software needs to run on it. The datagram itself may be discarded (an ICMP port-unreachable response is normal and harmless).

The HLC sends no other unsolicited traffic to any address.

4.3 IO LED annunciation

Visible health indication requiring no client. Blink period 500 ms.

LED	OCU	Safety mode	Meaning
Solid Orange	—	—	Technician mode, **or** within 60 s of boot
Solid Green	Connected	Active	Normal, safe
Blinking Green	Connected	Not active	OCU healthy, not safe
Blinking Red	Disconnected	Active	OCU unreachable, safe
Solid Red	Disconnected	Not active	OCU unreachable and not safe

Suspended while a manual colour is set via `set_io_led` (section 4.6).

4.4 Serial diagnostic stream

115 200 baud, 8-N-1, USB CDC, **output only** — the HLC accepts no serial commands. Output is suppressed entirely when no USB host has opened the port, so an unattended board spends no time formatting it.

Once per second the complete status document is printed as `name: value` lines, grouped under `[config]`, `[overview]`, `[imu]`, `[gps]`:

```None
========== DEVICE STATUS ==========
  localIp: 192.168.1.198
  type: "status"
  firmwareVersion: "1.5.1"
  ...
[config]
  imuPitchAxis: "X"
  ...
[overview]
  relays_status: [false,false,false,false]
  ocuConnected: true
  ...
===================================
```

`localIp` is the only line not present in the API document — it is the board's **actual link address**, whereas `controllerIp` is the *configured* one. A mismatch indicates a network configuration problem.

OTA errors appear only here, never over the API, formatted OTA Error[<code>]: <message>.

---

5. Data dictionary

5.1 Status document structure

```None
{
  "type": "status",
  "ledInternal": false,
  "ledIo": "GREEN",
  "button_tech": false,
  "motorWorkHours": 12.5,
  "motorWorkSeconds": 45000,
  "config":   { ... },
  "overview": { ... },
  "imu":      { ... },
  "gps":      { ... }
}
```

5.2 `config`

Field	Type	Unit / Range	Meaning
`firmwareVersion`	string	—	Firmware version
`controllerIp`	string	dotted quad	Configured IP
`whitelistIps`	string[ ]	0–3	Permitted client IPs
`technicianMode`	bool	—	Technician mode active
`burnedHours`	number	hours, 2 dp	Operating time since serial-number burn
`sessionHours`	number	hours, 2 dp	Uptime since last boot
`techLedColor`	string	OFF/GREEN/ RED/ORANGE	Current IO LED colour
`imuPitchAxis` / `imuRollAxis` / `imuYawAxis`	string	X/Y/Z	Sensor axis carrying each angle
`imuPitchInvert` / `imuRollInvert` / `imuYawInvert`	bool	—	Axis negated

5.3 `overview`

Field	Type	Unit / Range	Meaning
`powerConnected`	bool	—	INA219 monitor present
`powerSane`	bool	—	Power readings pass sanity check
`systemVoltage`	number	V — **`-1` = absent**	System bus voltage
`systemCurrent_A`	number	A — **`-1` = absent**	System bus current
`relays_status`	bool[4]	—	Relay states, index 0–3
`optoin_status`	bool[4]	—	Opto input states; [0],[1] = EPC safety, [2],[3] reserved
`safetyMode`	bool	—	`optoin_status [0] AND [1]`
`safetyModeDurationMs`	number	ms — 0 while safe	Continuous unsafe duration
`ocuConnected`	bool	—	OCU reachable
`ocuDisconnectedDurationMs`	number	ms — 0 while connected	Continuous unreachable duration

5.4 `imu`

Field	Type	Unit / Range	Meaning
`pitch`, `roll`, `yaw`	number	degrees, (−180, 180)	Calculated orientation
`angleSane`	bool	—	Orientation passes sanity check
`imuValid`, `imu2Valid`	bool	—	I2C read succeeded
`imu1Sane`, `imu2Sane`	bool	—	IMU responding, not all-zero, not frozen
`imuX/Y/Z`, `imu2X/Y/Z`	number	g	Linear acceleration
`imuGx/Gy/Gz`, `imu2Gx/Gy/Gz`	number	°/s	Angular rate, calibration offset removed
`imuTemp`	number	°C	IMU die temperature, averaged over readable units

5.5 `gps`

Field	Type	Unit / Range	Meaning
`gpsConnected`	bool	—	Receiver communicating
`gpsSane`	bool	—	Position passes sanity check
`gpsSatellites`	number	count	Satellites in fix
`gpsLat`, `gpsLng`	number	degrees — **0 when fix lost**	Current position
`gpsAlt`	number	m — **0 when fix lost**	Current altitude
`lastGpsLat/Lng/Alt`	number	degrees / m	Last known-good, **retained** on fix loss
`gpsHeading`	number	degrees 0–360	Course over ground
`gpsGroundSpeed`	number	km/hr	Horizontal speed
`gpsSpeedNorth/East/Down`	number	km/hr	Velocity components
`gpsTime`	string	`YYYY-MM-DD hh:mm:ss` — **empty on no fix**	GNSS UTC time

6. Fault detection and health monitoring

6.1 Health flags

Every sensor group carries a two-level indication: *is it talking* and *is what it says believable*.

Flag	False means
`imuValid` / `imu2Valid`	That IMU's I2C read failed — not communicating
`imu1Sane` / `imu2Sane`	That IMU is not communicating, **or** its data is frozen or all-zero. This is a **long-run** health signal: vehicle dynamics can never momentarily flip it false.
`angleSane`	The computed orientation is frozen or all-zero, or no IMU is sane
`gpsConnected`	GNSS receiver not communicating
`gpsSane`	No valid fix, or position frozen/all-zero
`powerConnected`	INA219 power monitor not detected at boot
`powerSane`	Power monitor absent, or readings frozen, all-zero, or negative
`ocuConnected`	OCU has not been confirmed reachable within 5 s

How "sane" is determined. A reading group fails if *every* value in it is zero, or if the whole group has been **byte-identical for 3 consecutive updates** (100 Hz sampling ⇒ ~30 ms) — real sensor noise means live readings essentially never repeat exactly, so this reliably catches a frozen or disconnected sensor. The power monitor additionally fails on any negative value.

`imu1Sane` / `imu2Sane` select which IMU feeds the orientation filter:

State	Behaviour
Both sane	Fused (lower noise)
One sane	That IMU alone
Neither sane	**Angles frozen at last values**

These flags reflect sustained sensor health only. They do **not** react to vehicle dynamics, so a hard manoeuvre never causes a transient false — any false reading indicates a genuine sensor problem.

6.2 Protocol-level errors

Error responses are always {"type":"error","code":"<id>","message":"<text>"}. **The code is stable across releases; the message text is not.** Clients should branch on code.

Code	HTTP	Message	Cause / client action
E-100	400	Request timeout	Body not sent within 100 ms — retry
E-101	401	Token required.	Add token
E-102	401	Invalid or expired token.	Re-login, retry
E-103	403	IP not allowed	Source IP not whitelisted — configuration error
E-110	200	Invalid username	Login rejected (returned in login_result)
E-111	200	Invalid password	Login rejected (returned in login_result)
E-200	200	Unknown request type	Unrecognised type — check spelling
E-201	200	Invalid relay number	relay_id outside 0–3
E-202	200	Invalid LED color	Colour not in permitted set
E-203	200	Invalid IP address or whitelist entry provided.	Malformed address, or more than 3 whitelist entries
E-204	200	Invalid IMU axis (expected X, Y or Z)	Bad axis token
E-205	200	Pitch, roll and yaw must each use a different axis	Axis map not a permutation

Note the split: transport/auth failures use HTTP status codes; application-level rejections return **HTTP 200** with an error body. A client that checks only the HTTP status will treat rejected commands as successful.

6.3 Elapsed-time counters

Two fields distinguish a momentary glitch from a sustained fault:

Field	Use
`ocuDisconnectedDurationMs`	How long the OCU has been continuously unreachable (0 while connected)
`safetyModeDurationMs`	How long the system has been continuously unsafe (0 while safe)

Both are `millis()`-based and reset on reboot. A value that stops increasing between polls while the condition persists indicates the HLC has stopped updating.

6.4 Out-of-band indication

Channel	Detects
IO LED (section 5.3)	OCU connectivity and safety state, visible with no client attached
Serial console (section 5.4)	Full state at 1 Hz, independent of the network; `localIp` vs `controllerIp` mismatch reveals network misconfiguration
Connection refused / timeout	HLC powered down, wrong IP, or link fault
HTTP 403 on every request	Client IP not whitelisted

6.5 Recommended client health check

```None
poll get_status every N ms
  ── transport
     no response .......................... HLC down / link fault
     HTTP 403 ............................. IP not whitelisted
     HTTP 401 ............................. re-login, retry
     HTTP 200 + type=="error" ............. command rejected —
read message

  ── liveness  (the HLC has no heartbeat; derive one)
     sessionHours not increasing .......... firmware stalled or
rebooting
     sessionHours went backwards .......... unit rebooted —
tokens invalid,
                                            technicianMode
cleared
  ── sensors
     !imuValid && !imu2Valid ............. NO orientation
updates; pitch/roll/
                                            yaw are STALE but
still populated
     !imu1Sane && !imu2Sane (sustained) ... degraded — coasting
on gyro
     !gpsSane or gpsSatellites low ........ position unreliable;
yaw uncorrected
     !powerConnected ...................... power telemetry
unavailable (-1)

  ── system
     !ocuConnected ........................ check
ocuDisconnectedDurationMs
     !safetyMode .......................... check
safetyModeDurationMs
```

6.6 Blind spots — what cannot be detected

These failure modes produce **no error indication**. A client must not assume that absence of an error means the data is correct.

Failure	Symptom	Why it is missed
Both IMUs fail	`pitch`/`roll`/`yaw` keep being reported, frozen at their last values	The angle fields are always populated. Only `imuValid`/`imu2Valid`/`*Sane` reveal it — the angles themselves look normal
An IMU is live but wrong (loose mount, drifted bias)	Plausible but incorrect angles; fused silently with the good unit	The sanity check detects *frozen* and *all-zero* data, not *wrong* data. The two IMUs are not cross-checked against each other
Yaw while stationary	Drifts without bound (~60°/min)	GPS correction is gated off below 3 km/hr by design. No flag marks yaw as uncorrected
Yaw while reversing	180° in error	GPS reports course over ground, not vehicle heading. The HLC has no reverse indication
Pitch/roll while cornering	Up to ~17° error at 0.3 g	The accelerometer cannot separate gravity from vehicle acceleration, and no compensation is applied
Gyro calibration corrupted at boot	Persistent bias for the whole session	Calibration requires the unit to be still and level; movement during the ~1 s window is not detected
Reboot	Tokens invalid, technician mode cleared, counters reset	No reboot-cause or boot-count field. Detect via `sessionHours` going backwards

There is no error/event log, no fault counter, no reboot-cause register and no watchdog. All diagnosis is by polling the flags above and by observing the serial console.

7. Data-quality notes for integrators

Orientation accuracy is condition-dependent. Summary:

Condition	Pitch / Roll	Yaw
Stationary	≈0.8–1.1°, bounded	**Unbounded drift**, ≈60°/min
Moving, GNSS fix	Up to ≈17° during 0.3 g manoeuvres	≈5° bounded; **180° wrong in reverse**
Moving, no fix	As above	**Unbounded drift**

Additionally:

- `pitch`/`roll` are referenced to the **attitude at power-on**, not to true level.
- `yaw` becomes an absolute bearing only after the vehicle has first moved at ≥3 km/hr with a fix.
- Telemetry is sampled on a fixed 10 ms cadence; API responses are at most 10 ms old. Polling faster than 100 Hz returns duplicate samples.

---

8. Appendix — defaults and acronyms

Parameter	Default
IP address	`192.168.1.198`
Subnet mask	`255.255.0.0`
Gateway	`192.168.<3rd octet of IP>.1`
API / OTA port	TCP 80 / TCP 65280
OCU probe port	UDP 5001 → 5001
Serial console	115 200 baud, 8-N-1
Login	`admin` / `1234`
Whitelist	`192.168.1.20`, `192.168.1.169`, `192.168.1.33`
OCU address	whitelisted entry ending `.169`
IMU axis map	Pitch=X, Roll=Y, Yaw=Z, no inversion

Acronym	Definition
ARP	Address Resolution Protocol
EPC	Endpoint Controller
GNSS	Global Navigation Satellite System
HLC	High-Level Control
ICD	Interface Control Document
IMU	Inertial Measurement Unit
OCU	Operator Control Unit
OTA	Over-The-Air (firmware update)